Variants of Gradient Descent

Chapter 4: **Training Models**
Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, Gurelien Geron, 3e, O'Reilly Media, 2022.

Epoch

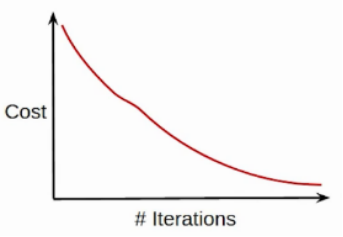
- An epoch is when an entire dataset passes through the network a single time (corresponds to one iteration in gradient descent).
- Number of epochs is the number of times the entire training dataset is shown to the network while training.
- Generally the more the epochs, the better the model fit.
- After a certain epochs, the model starts overfitting.
- Increase the number of epochs until the validation accuracy starts decreasing even when training accuracy is increasing (overfitting).

Batch

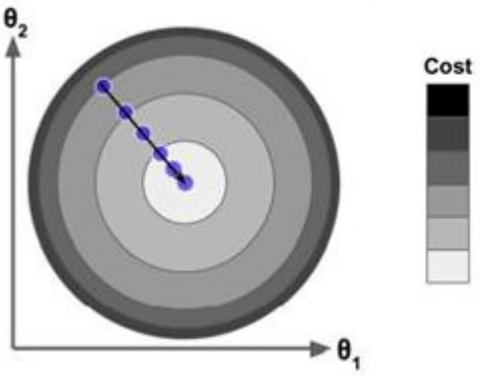
- If we have a powerful machine, or less data, the whole data can be processed at once.
- Otherwise, split the dataset into pieces and process one piece at a time.
- A single piece is called a batch.
- So in this case multiple iterations will be required to complete 1 epoch.
- The batch size is a hyperparameter that defines the number of samples to work through before updating the internal model parameters.
- Bigger batch sizes learn faster but require more memory space.
- A training dataset can be divided into one or more batches.
- A good default for batch size might be 32. Also try 64, 128, 256, and so on.

1. Batch Gradient Descent

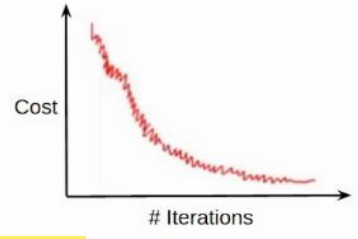
- When all training samples are used to create one batch, the learning algorithm is called batch gradient descent.
- Batch Size = n, where n is the size of the training set.
- 1 Epoch = 1 Batch.
- Weight update: $\theta = \theta - \alpha \sum_{i=1}^{n} \frac{d}{d\theta} J(\theta)$
- Computational cost per iteration / epoch is very high as entire data set is loaded at a time, and entire dataset needs to be traversed for each update.
- Cost function reduces smoothly, hence converges in lesser number of steps.



- The vanilla gradient descent version involves calculations over the full training set X, at each Gradient Descent step.
- This is why the algorithm is called Batch Gradient Descent: it uses the whole batch of training data at every step.
- As a result it is terribly slow on very large training sets.
- However, Gradient Descent scales well with the number of features; training a Linear Regression model when there are hundreds of thousands of features is much faster using Gradient Descent than using the Normal Equation or SVD decomposition.

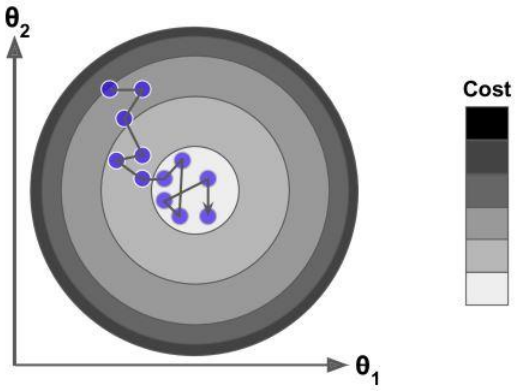


2. Stochastic Gradient Descent (SGD)



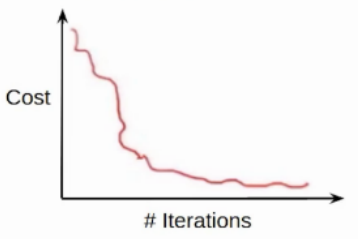
- When the batch is the size of one sample, the learning algorithm is called stochastic gradient descent.
- Batch Size = 1.
- 1 Epoch = n Batches, where n is the size of the training set.
- Weight update: $\theta = \theta - \alpha \frac{d}{d\theta} J(\theta)$
- Not computationally expensive for a single batch or iteration, but takes more time for an epoch.
- Cost function exhibits lots of variations, hence takes more time to converge.

- The main problem with Batch Gradient Descent is the fact that it uses the whole training set to compute the gradients at every step, which makes it very slow when the training set is large.
- At the opposite extreme, Stochastic Gradient Descent just picks a *random* instance in the training set at every step and computes the gradients based only on that single instance.
- Obviously this makes the algorithm much faster per iteration since it has very little data to manipulate at every iteration.
- It also makes it possible to train on huge training sets, since only one instance needs to be in memory at each iteration. Hence SGD can be implemented as an out-of-core algorithm.
- On the other hand, due to its stochastic (i.e., random) nature, this algorithm is much less regular than Batch Gradient Descent.
- Over time it will end up very close to the minimum, but once it gets there it will continue to bounce around, never settling down. So once the algorithm stops, the final parameter values are good, but not optimal.



- When the cost function is very irregular, this can actually help the algorithm jump out of local minima, so Stochastic Gradient Descent has a better chance of finding the global minimum than Batch Gradient Descent does.
- Therefore randomness is good to escape from local optima, but bad because it means that the algorithm can never settle at the minimum.

3. Mini-batch Gradient Descent



- When the batch size is more than one sample and less than the size of the training dataset, the learning algorithm is called mini-batch gradient descent.
- 1 < Batch Size < Size of Training Set.
- 1 Epoch = number of batches = n / batch size, where n is the size of the training set.
- Weight update: $\theta = \theta - \alpha \sum_{i=1}^{batch\ size} \frac{d}{d\theta} J(\theta)$
- Computation time per epoch is lesser as compared to SDG.
- Computational cost per iteration is lesser as compared to batch gradient descent.
- Smoother cost function as compared to SDG.

Gradient Descent Optimization

- Three possible ways to optimize Gradient Descent are:
  - **Adapt the gradient component ($\partial J(\theta)/\partial\theta_j$):**
    - It adds history to the parameter update equation based on the gradient encountered in the previous updates.
    - It adds a momentum term of the update vector of the past time step to the current update vector.
    - This change is based on the metaphor of momentum from physics where acceleration in a direction can be accumulated from past updates.
    - It accelerates the optimization process.
    - Instead of using only one single gradient like in stochastic vanilla gradient descent to update the weight, take an aggregate of multiple gradients. Specifically, these optimizers use the exponential moving average of gradients, which accelerates the algorithm.
  - **Adapt the learning rate component (α):** Instead of keeping a constant learning rate, adapt the learning rate according to the magnitude of the gradient(s).
  - **Adapt both the gradient component and the learning rate component.**
- Some optimized versions are:
  - Gradient Descent with Momentum (gradient)
  - Adaptive Gradient - AdaGrad (learning rate)
  - RMSprop (learning rate)
  - Adadelta (learning rate)
  - Nesterov Accelerated Gradient - NAG (gradient)
  - Adam (learning rate, gradient)
  - Nadam (learning rate, gradient)
  - AMSGrad (learning rate, gradient)